

3. Flowchart dan Pseudocode

3.1 Tujuan Praktikum

Setelah mempelajari Bab ini, mahasiswa diharapkan dapat:

1. Dapat menjelaskan dan menggunakan flowchart dengan benar.
2. Dapat menjelaskan dan menggunakan pseudocode dengan benar.

3.2 Alat dan Bahan

Praktikum ini membutuhkan perangkat komputer yang memiliki spesifikasi minimum sebagai berikut:

1. Terkoneksi ke Internet dan dapat mengunduh package-package Python.
2. Mampu menjalankan sistem operasi Windows 10 atau Ubuntu Linux.

Perangkat lunak yang diperlukan untuk mendukung praktikum ini adalah sebagai berikut:

1. Aplikasi draw.io atau diagram.net yang diakses melalui <https://app.diagrams.net/>.
2. Python 3.7 atau 3.8 yang terinstall menggunakan Anaconda atau Installer Python lainnya.
3. Editor Visual Studio Code, Repl Python 3 (online), pyCharm, atau Spyder

3.3 Materi

3.3.1 Algoritma

Algoritma biasanya didefinisikan sebagai rangkaian terurut langkah-langkah yang logis dan sistematis yang disusun untuk menyelesaikan suatu masalah. Tujuan algoritma adalah memberikan petunjuk tentang langkah-langkah logika penyelesaian masalah dalam bentuk yang mudah dipahami nalar manusia sebagai acuan yang membantu dalam mengembangkan program komputer. Pemahaman terhadap algoritma akan mencegah sejak dini kemungkinan terjadinya kesalahan logika pada program komputer yang dikembangkan.

Penulisan (Notasi Algoritma)

Ada tiga macam bentuk notasi algoritma antara lain:

1. Uraian deskriptif
2. *Flowchart* / Diagram Alir
3. *Pseudocode*

3.3.2 Uraian Deskriptif

Contoh: menyelesaikan permasalahan menghitung luas dan keliling suatu lingkaran Algoritma Hitung_Luas_dan_Keliling_Lingkaran.

Deskripsi:

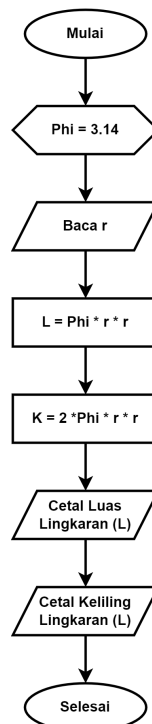
1. Masukkan jari-jari lingkaran (r).
2. Hitung luas lingkaran dengan rumus $L = p * r ** 2$.
3. Hitung keliling lingkaran dengan rumus $K = 2 * p * r$.
4. Tampilkan luas lingkaran.
5. Tampilkan keliling lingkaran.

3.3.3 Flowchart / Diagram Alir

Flowchart adalah gambaran dalam bentuk diagram alir dari algoritma algoritma dalam suatu program, yang menyatakan arah alur program tersebut.

Contoh:

Menghitung luas dan keliling lingkaran yang algoritmanya dinotasikan dalam bentuk diagram alir (*flowchart*).



Gambar 3.1: Flowchart mencari luas dan keliling lingkaran.

Flowchart menolong analis dan programmer untuk memecahkan masalah kedalam segmen-segmen yang lebih kecil dan menolong dalam menganalisis alternatif-alternatif lain dalam pengoperasian.

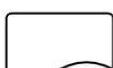
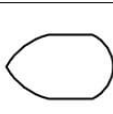
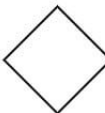
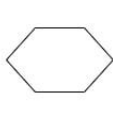
Kegunaan:

1. Untuk mendesain program.
2. Untuk merepresentasikan program.

Maka, flowchart harus dapat merepresentasikan komponen-komponen dalam bahasa pemrograman.

3.3.4 Notasi Flowchart

Pada dasarnya notasi dalam flowchart memiliki arti yang berbeda-beda. Berikut adalah notasi yang sering digunakan dalam proses pembuatan flowchart.

	Flow Simbol yang digunakan untuk menggabungkan antara simbol yang satu dengan simbol yang lain. Simbol ini disebut juga dengan Connecting Line.		Input/output Simbol yang menyatakan proses input atau output tanpa tergantung peralatan.
	On-Page Reference Simbol untuk keluar - masuk atau penyambungan proses dalam lembar kerja yang sama.		Manual Operation Simbol yang menyatakan suatu proses yang tidak dilakukan oleh komputer.
	Off-Page Reference Simbol untuk keluar - masuk atau penyambungan proses dalam lembar kerja yang berbeda.		Document Simbol yang menyatakan bahwa input berasal dari dokumen dalam bentuk fisik, atau output yang perlu dicetak.
	Terminator Simbol yang menyatakan awal atau akhir suatu program.		Predefine Proses Simbol untuk pelaksanaan suatu bagian (sub-program) atau prosedur.
	Process Simbol yang menyatakan suatu proses yang dilakukan komputer.		Display Simbol yang menyatakan peralatan output yang digunakan.
	Decision Simbol yang menunjukan kondisi tertentu yang akan menghasilkan dua kemungkinan jawaban, yaitu ya dan tidak.		Preparation Simbol yang menyatakan penyediaan tempat penyimpanan suatu pengolahan untuk memberikan nilai awal.

Gambar 3.2: Notasi Flowchart.

Notasi di atas memiliki jenis dan fungsi yang berbeda-beda. Ada yang berfungsi untuk menghubungkan satu notasi dengan notasi lainnya seperti notasi flow, on-page dan off-page reference. Selain itu ada juga notasi yang berfungsi untuk menunjukan suatu proses yang sedang berjalan, dan yang terakhir terdapat notasi yang berfungsi untuk memasukan input dan menampilkan output.

3.3.5 Pseudocode

Notasi yang menyerupai notasi Bahasa pemrograman tingkat tinggi, misalnya Bahasa C dan Python. Struktur algoritma dibagi ke dalam beberapa bagian, diantaranya:

1. Bagian kepala (header).
2. Bagian Deklarasi (definisi variable).
3. Bagian Deskripsi (rincian langkah).

Contoh:

```

Algoritma Luas_persegi_panjang
    {Menghitung sebuah luas persegipanjang
  
```

apabila panjang dan lebar persegi panjang
tersebut diberikan}

Deklarasi

{Definisi nama peubah/variabel}

`float` panjang, lebar, luas

Deskripsi

`READ (panjang,lebar)`

#bisa juga : INPUT

`luas <- panjang * lebar`

`WRITE (Luas)`

#bisa juga : OUTPUT

3.3.6 Notasi Pseudocode

Berikut ini notasi yang sering digunakan dalam pseudocode

1. **INPUT** -> Digunakan untuk menunjukkan proses memasukan suatu isi variabel.
2. **OUTPUT** -> Digunakan untuk menunjukkan proses keluaran yang terjadi.
3. **WHILE** -> Digunakan untuk sebuah perulangan yang memiliki iterasi awali.
4. **FOR** -> Digunakan untuk sebuah perulangan perhitungan iterasi.
5. **REPEAT - UNTIL** -> Digunakan untuk sebuah perulangan yang memiliki kondisi akhir.
6. **IF – THEN – ELSE** -> Digunakan untuk mengambil sebuah keputusan dari beberapa kondisi.

3.4 Kegiatan Praktikum

Kegiatan praktikum yang akan dilakukan adalah sebagai berikut:

1. Membuat Flowchart.
2. Membuat Pseudocode.

3.4.1 Membuat Flowchart

Kasus: Ganjil Genap.

Buatkan flowchart untuk menentukan bilangan yang diinputkan adalah ganjil atau genap.
Gambar flowchart lengkap flowchart dapat dilihat pada gambar 3.3.

Pembahasan:

1. Pertama pengguna menginput data yang berupa nilai dari bilangan bulat.
2. Kemudian nilai yang dimasukan diproses dengan cara dibagi dengan angka 2.
3. Jika sisa bagi sama dengan 0 berarti bilangan yang dimasukan adalah bilangan genap.
4. Jika sisa bagi tidak sama dengan 0 berarti bilangan yang dimasukan adalah bilangan ganjil.
5. Selesai.

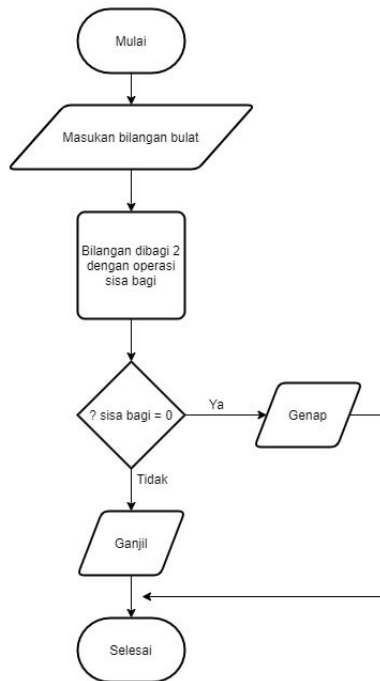
3.4.2 Membuat Pseudocode

Kasus: Ganjil Genap.

Pseudocode untuk menentukan apakah bilangan yang dimasukan ganjil atau genap.

Pembahasan:

1. Pertama pengguna menginput data yang berupa nilai dari bilangan bulat.
2. Kemudian nilai yang dimasukan diproses dengan cara dibagi dengan angka 2.
3. Jika sisa bagi sama dengan 0 berarti bilangan yang dimasukan adalah bilangan genap.
4. Jika sisa bagi tidak sama dengan 0 berarti bilangan yang dimasukan adalah bilangan ganjil.
5. Selesai.



Gambar 3.3: Flowchart Ganjil Genap.

Program ganjil_genap

Deklarasi

var number : integer

Deskripsi

INPUT number

IF (number modulus 2 == 0) THEN

 OUTPUT "genap"

ELSE

 OUTPUT "ganjil"

3.5 Latihan Mandiri

Latihan 3.1 Buatlah algoritma untuk menentukan apakah suatu bilangan merupakan bilangan prima atau bukan. Algoritma yang dibuat disajikan dalam flowchart dan pseudocode. ■

Latihan 3.2 Buatlah algoritma untuk menentukan jenis akar dari suatu persamaan kuadrat. Jenis akar persamaan kuadrat sebagai berikut:

1. $D \geq 0$; Akar Real/Nyata.
2. $D > 0$; Akar Real dan Berlainan.
3. $D = 0$; Akar Real dan Kembar.
4. $D < 0$; Akar Imajiner/Tidak Real/Khayal.
5. Selesai.

Algoritma yang dibuat disajikan dalam flowchart dan pseudocode. ■

Latihan 3.3 Buatlah algoritma untuk menghitung nilai faktorial dari suatu bilangan. Algoritma yang dibuat disajikan dalam flowchart dan pseudocode. ■